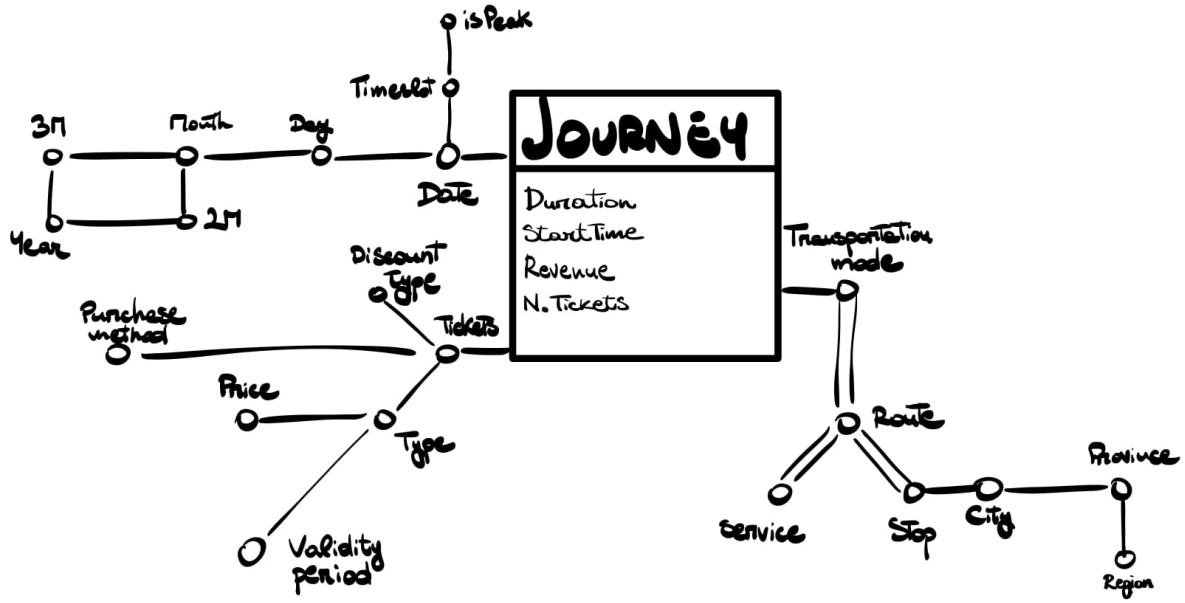
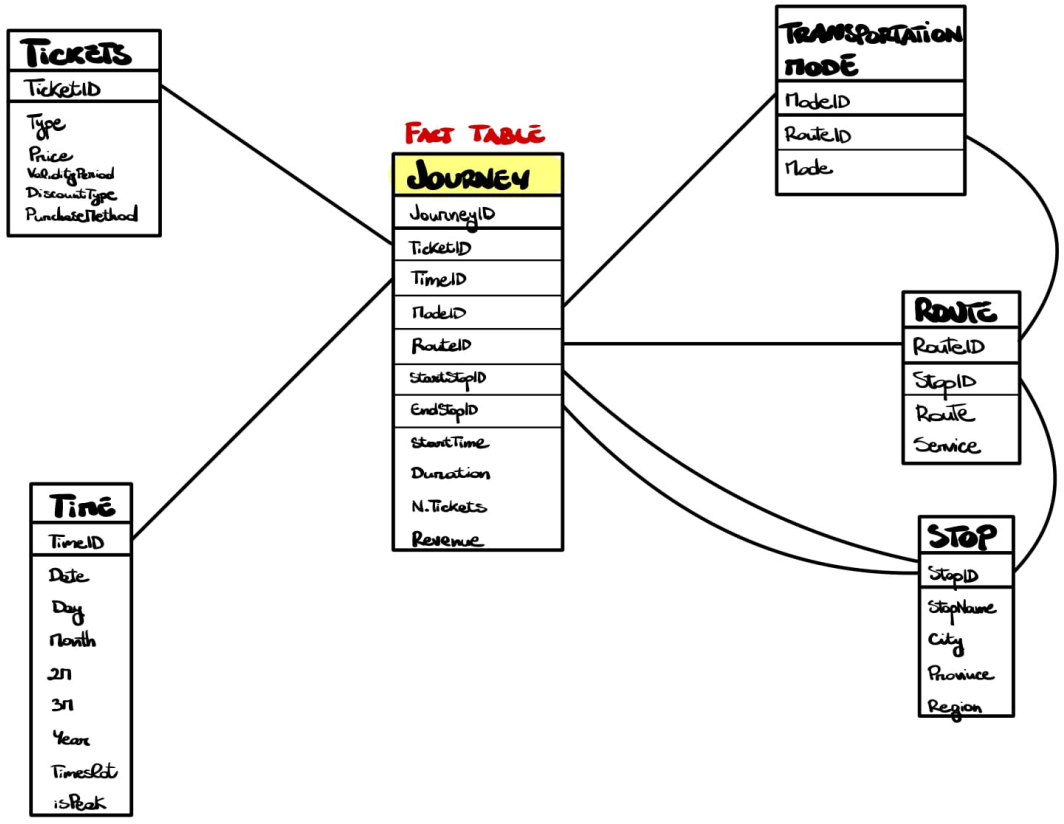
homework_1

Assignment

https://prod-files-secure.s3.us-west-2.amazonaws.com/c8bf098b-43ce-4bd8-ab7e-5fb6c3a28a08/89c4fc5f-6cfc-42a4-be21-ac7a1946359d/Homework-1.pdf

Task 1





Task 2

Write the following frequent queries using the extended SQL language.

Point A

Separately for each transportation mode and for each month of the year, analyze: the average daily number of tickets, the cumulative number of tickets from the beginning of the year, and the percentage of tickets using each transportation mode over the total number of tickets in that month.

```
SELECT Mode, Month, Year,
    SUM(N_Tickets)/COUNT(DISTINCT Day),
    SUM(SUM(N_Tickets)) OVER (PARTITION BY Year
                                                    ORDER BY Month
                                                    ROWS UNBOUNDED PRECEDING),
    SUM(N_Tickets)/SUM(SUM(N_Tickets))*100 OVER (PARTITION BY Mode, Month

FROM Journey J, Time T, TransportationMode TM
WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
GROUP BY Mode, Month, Year;
```

Point B

Considering journeys from 2022, separately for each mode and city, analyze: the average journey duration, the total revenue generated from that city, the percentage of total revenue contributed by each route for the corresponding mode in a city, and assign a rank to each route within its transportation mode based on the total revenue generated in decreasing order.

```
SELECT Mode, City, Route,
    AVG(Duration),
    SUM(SUM(Revenue)) OVER (PARTITION BY City),
    SUM(Revenue)/SUM(SUM(Revenue)) OVER (PARTITION BY Route, Mode, City),
    RANK() OVER (PARTITION BY Route, Mode
                            ORDER BY SUM(Revenue) DESC)
FROM Journey J, TransportationMode TM, Time T, Stop S, Route R
WHERE J.ModeID = TM.ModeID AND J.TimeID = T.TimeID AND J.StartStopID = S.StopID AND J.Ro
    AND T.Year = '2022'
GROUP BY Mode, City, Route;
```

Task 3

Create and update a materialized view with CREATE MATERIALIZED VIEW and CREATE MATERIALIZED VIEW LOG in ORACLE.

Point A

Separately for each transportation mode and for each month, analyze the average daily number of tickets.

```
INSERT INTO ViewAvgDailyTickets (Mode, Month, Year, AvgDailyTickets)
(SELECT Mode, Month, Year, SUM(N_Tickets)/COUNT(DISTINCT Day)
 FROM Journey J, Time T, TransportationMode TM
 WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
 GROUP BY Mode, Month, Year);
```

Point B

Separately for each transportation mode and for each month, analyze the cumulative number of tickets from the beginning of the year.

```
INSERT INTO ViewCumulativeNTickets (Mode, Month, Year, CumulativeNTickets)
(SELECT Mode, Month, Year,
     SUM(SUM(N_Tickets)) OVER (PARTITION BY Year
                                                    ORDER BY Month
                                                    ROWS UNBOUNDED PRECEDING),
 FROM Journey J, Time T, TransportationMode TM
 WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
 GROUP BY Mode, Month, Year);
```

Point C

Separately for each transportation mode and for each month, analyze the total number of tickets sold, the total revenue, and the average revenue.

```
INSERT INTO ViewRevenue (Mode, Month, Year, TotalNTickets, TotalRevenue, AvgRevenue)
(SELECT Mode, Month, Year,
     SUM(N_Tickets),
     SUM(Revenue),
     SUM(Revenue)/SUM(N_Tickets)
 FROM Journey J, Time T, TransportationMode TM
 WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
 GROUP BY Mode, Month, Year);
```

Point D

Separately for each transportation mode and for each month, analyze the total number of tickets sold, the total revenue, and the average revenue for the year 2024.

```
INSERT INTO ViewRevenue (Mode, Month, Year, TotalNTickets, TotalRevenue, AvgRevenue)
(SELECT Mode, Month, Year,
     SUM(N_Tickets),
     SUM(Revenue),
     SUM(Revenue)/SUM(N_Tickets)
 FROM Journey J, Time T, TransportationMode TM
 WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID AND T.Year = '2024'
 GROUP BY Mode, Month, Year);
```

Point E

Analyze the percentage of tickets related to each transportation mode and month over the total number of tickets of the month for each transportation mode.

1. Define a materialized view with CREATE MATERIALIZED VIEW useful to reduce the response time of the reported frequent queries.

   ```
   CREATE MATERIALIZED VIEW ViewTickets
   BUILD IMMEDIATE
   REFRESH FAST ON COMMIT AS
   SELECT Mode, Month, Year,
       SUM(N_Tickets),
       SUM(N_Tickets)/SUM(SUM(N_Tickets)) OVER (PARTITION BY Month, Year)
   FROM Journey J, Time T, TransportationMode TM
   WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
   GROUP BY TM.Mode, T.Month, T.Year;
   ```

2. Define the materialized view logs with CREATE MATERIALIZED VIEW LOG for each table where you deem it necessary. For which tables is it useful to keep track of logs? Identify all and only the necessary tables. Furthermore, for each table identify all and only the attributes for which it is necessary to keep track of the variations.

   ```
   CREATE MATERIALIZED VIEW LOG ON Journey
   WITH SEQUENCE, ROWID
   (TimeID, ModeID, N_Tickets)
   INCLUDING NEW VALUES;

   CREATE MATERIALIZED VIEW LOG ON Time
   WITH SEQUENCE, ROWID
   (Month, Year)
   INCLUDING NEW VALUES;

   CREATE MATERIALIZED VIEW LOG ON TransportationMode
   WITH SEQUENCE, ROWID
   (Mode)
   INCLUDING NEW VALUES;
   ```

3. Specify which operations (e.g. INSERT on a specific table) cause an update of the defined MATERIALIZED VIEW.

> INSERT, UPDATE, or DELETE operations on the `Journey`, `Time`, and `TransportationMode` tables will trigger an update of the defined materialized view.

Task 4

Update and management of views via Trigger assuming that the CREATE MATERIALIZED VIEW command is not available, create the materialized views defined in the previous exercise and define the update procedure starting from changes on the fact table created by means of a trigger.

Point A

Create the structure of the materialized view with CREATE TABLE VM1 (...).

```
CREATE TABLE VM1(
    Mode VARCHAR(20),
    Month VARCHAR(20),
    Year INTEGER,
    TotalTickets INTEGER,
            CHECK(TotalTickets IS NOT NULL AND TotalTickets > 0)
    PercentageTickets FLOAT,
            CHECK(PercentageTickets IS NOT NULL AND PercentageTickets > 0)
PRIMARY KEY(Mode, Month, Year)
)
```

Point B

Specify an example of statement to populate the VM1 table with the necessary records using the statement INSERT INTO VM1 (...) (SELECT ...).

```
INSERT INTO VM1(Mode, Month, Year, TotalTickets, PercentageTickets)
(SELECT Mode, Month, Year,
     SUM(N_Tickets),
     SUM(N_Tickets)/SUM(SUM(N_Tickets)) OVER (PARTITION BY Month, Year)
 FROM Journey J, Time T, TransportationMode TM
 WHERE J.TimeID = T.TimeID AND J.ModeID = TM.ModeID
 GROUP BY TM.Mode, T.Month, T.Year)
```

Point C

Write the triggers necessary to propagate the changes (insertion of a new record) made in the FACTS table to the materialized view VM1.

```
CREATE TRIGGER RefreshViewTickets
AFTER UPDATE OF TimeID, ModeID, N_Tickets ON Journey
FOR EACH ROW
DECLARE
    varMonth, varMode VARCHAR(20);
    varYear, varTotalTickets, varFullTotalTickets INTEGER;
    N NUMBER;
BEGIN
    SELECT Month, Year INTO varMonth, varYear
    FROM Time
    WHERE TimeID=:NEW.TimeID
```

```
    SELECT Mode INTO varMode
    FROM TransportationMode
    WHERE ModeID=:NEW.ModeID

    SELECT SUM(N_Tickets) INTO varTotalTickets
    FROM Journey J, Time T
    WHERE J.ModeID=:NEW.ModeID AND J.TimeID=T.TimeID
    AND T.Month=varMonth AND T.Year=varYear

    SELECT SUM(N_Tickets) INTO varFullTotalTickets
    FROM Journey J, Time T
    WHERE J.TimeID=T.TimeID AND T.Month=varMonth AND T.Year=varYear

    SELECT COUNT(*) INTO N
    FROM VM1
    WHERE Month=varMonth AND Year=varYear AND Mode=varMode
    AND TotalTickets=varTotalTickets AND PercentageTickets=varPercentageTickets

    IF(N>0) THEN
        UPDATE VM1
        SET TotalTickets=varTotalTickets,
                PercentageTickets=varTotalTickets/varFullTotalTickets
        WHERE Month=varMonth AND Year=varYear AND Mode=varMode
    ELSE
        INSERT INTO VM1(Mode, Month, Year, TotalTickets, PercentageTickets)
        VALUES (varMonth, varYear, varMode, varTotalTickets, varTotalTickets/varFullTota
    END IF
END
```

Point D

Specify which operations (e.g. INSERT) trigger the trigger created in 4.c.

> Given the defined trigger at point C the operations trigger that will cause the activation are all INSERT, UPDATE and DELETE operations on `Journey`.